Demystifying the Lean Theorem Prover

Reconstructing the Curry-Howard Correspondence via Pure Functional Programming in Python

Anuj Jha

March 4, 2026

SLP: Prof. Sudarshan Vijay

Agenda

1. The Philosophy: Proofs as Programs
2. Simulating the Lean Kernel in Python
3. Case Study 1: Proving Addition
4. Case Study 2: Multiplication & Decomposition
5. Case Study 3: Inequality and Existentials
6. Insights into the Lean 4 Compiler Architecture

Conclusion

1. The Philosophy: Proofs as Programs

The Core Principle

"Functional programming is one input, one output per function."

My goal with this project was to understand how modern interactive theorem provers like Lean 4 actually work under the hood.

What I discovered is that **Lean's proof system is exactly this concept applied to mathematics.**

Instead of viewing a proof as a sequence of logical deduction steps, we view a tactic as a pure function:

$$\texttt{tactic} : \texttt{ProofState} \to \texttt{ProofState}$$

The Curry-Howard Correspondence

Why does this work? Because proving a mathematical theorem is literally identical to compiling a computer program.

Mathematics / Logic	**Type Theory / Programming**
Proposition (e.g. $2 + 2 = 4$)	Type
Proof	A valid value of that type
Tactic	Function that constructs the value
"Q.E.D." (Verification)	Type-Checker accepting the program

When Lean says "Goals accomplished," it just means the program type-checked successfully.

2. Simulating the Lean Kernel in Python

Expressions as Tagged Trees

In Lean 4, ALL mathematical expressions are represented dynamically as an `Expr` tree. Since Python lacks algebraic data types, I successfully simulated this using **immutable tagged tuples**.

```
# Constructors (pure functions that build syntax trees)
def succ(n):   return ("succ", n)                    # S(n)
def add(a, b): return ("add", a, b)                  # a + b
def eq(a, b):  return ("eq", a, b)                   # a = b
def le(a, b):  return ("le", a, b)                   # a <= b

# Destructors (For programmatic Pattern Matching)
def is_add(t):   return t[0] == "add"
def left_of(t):  return t[1]
def right_of(t): return t[2]
```

To construct the goal $2 + 2 = 4$:

```
goal = eq(add(two(), two()), four())
```

The Variable Naming Problem (Concept)

A major challenge in math engines is variable names. To a human, `fun x → x` is identical to `fun y → y`.

But to a computer, comparing the letters 'x' and 'y' means they look completely different!

The Solution: De Bruijn Indices

Instead of giving variables names, we just give them **directions**.

A variable simply becomes: *"I am the variable defined N layers above me."*

De Bruijn Indices: A Simple Visual

We replace names with "arrows" pointing up to their creator.

- `fun x` $\rightarrow$ `x` simply becomes `fun` $\rightarrow$ `(Variable 0 layers up)`
- `fun y` $\rightarrow$ `y` simply becomes `fun` $\rightarrow$ `(Variable 0 layers up)`

Because both functions use exactly the same directions, the computer instantly knows they are mathematically identical! We never have to worry about accidentally mixing up variable names again.

Handling Alpha-Equivalence: De Bruijn Indices

A major challenge in building a theorem prover is variable shadowing. How does the kernel know `fun x → x` is identical to `fun y → y`?

The solution I studied is the **De Bruijn Index System**: Instead of names, bounded variables (`BVar`) use numeric offsets pointing upward to the lambda that bounds them.

- `fun x → x` internally becomes `lam("x", BVar(0))`
- `fun y → y` internally becomes `lam("y", BVar(0))`

The Result: Structural hashing entirely ignores the pretty-printing names ("x", "y"). These trees hash to the exact same pointer in memory natively!

De Bruijn Indices in Nested Scopes

How does this scaling representation work for complex math with multiple variables? The index simply counts how many λ-binders you must "jump over" going up the AST.

Consider the standard math expression: `fun x → fun y → x (y x)`

De Bruijn internal representation:

```
lam("x", lam("y", App(BVar(1), App(BVar(0), BVar(1)))))
```

- `BVar(0)` points to the innermost binder (`y`).
- `BVar(1)` skips `y` and points to the outer binder (`x`).

This eliminates all alpha-renaming bugs from the Kernel entirely!

Rewrite Rules = Pattern-Matching Functions

I implemented rewrite rules strictly as purely functional mappings: `term` $\rightarrow$ `term | None`. They search for a structural pattern and construct a new AST branch.

```
def add_succ_rule(term):
    """ Implementing the Peano Axiom: (a + S(b)) -> S(a + b) """
    if is_add(term) and is_succ(right_of(term)):

        a = left_of(term)
        b = pred_of(right_of(term))

        # Return cleanly constructed new tuple
        return succ(add(a, b))
    return None
```

Notice there is no state mutation. This guarantees mathematical purity.

Tactics as Higher-Order Closures

To simulate a tactic like Lean's `rw`, I used **closures** that take a mathematical rule and return a state transition function.

```python
def nth_rewrite(n, rule, name):
    """
    Returns a closure that performs DFS traversal, applying the rule
    to exactly the Nth occurrence it finds.
    """
    def tactic(goal):
        return apply_nth(goal, rule, n)
    tactic._lean = f"nth_rewrite {n} [{name}]"

    # Returning the higher-order function composition
    return tactic
```

3. Case Study 1: Proving Addition

Proving $2 + 2 = 4$: The Initial State

Let's walk through how my simulation evaluates $2 + 2 = 4$, replicating Lean's behavior exactly.

Initial Goal Tree: `eq( add(2, 2), 4 )`

The DFS Trap:
If we just write `rw [two_eq_succ_one]`, the DFS traversal hits the VERY FIRST `2` it finds—the **left** operand.

Our tree becomes: `eq( add(S(1), 2), 4 )` $\implies S(1) + 2$

Proving $2 + 2 = 4$: The Peano Mismatch

Why is $S(1) + 2$ a problem?

Look at the Peano addition axiom (`add_succ`):

$$a + S(b) \to S(a + b)$$

This rule **ONLY** matches if the **right** operand has an S. It doesn't know what to do with an S on the **left**! If we unfold the left side, the proof gets completely stuck.

Proving $2 + 2 = 4$: The Solution

We must use `nth_rewrite 2`! This forces the engine to bypass the left operand and target the **second** DFS occurrence.

Step 1: `nth_rewrite(2, two_eq_succ_one)`

- Target the right operand: `eq( add(2, S(1)), 4 )`

Step 2: `rw(add_succ_rule)`

- The rule sees the S on the right, and "wraps" it around the addition!
- Tree becomes: `eq( succ(add(2, 1)), 4 )`

This exact peeling process continues 5 more times until we reach `succ(succ(2)) = 4`.

4. Case Study 2: Multiplication & Decomposition

Multiplication is Cascading Addition

Multiplication is defined by structural recursion strictly on the right operand:

$$a \times 0 = 0$$

$$a \times \mathrm{succ}(b) = (a \times b) + a$$

The axiom `mul_succ` forces multiplication to decompose into addition subproblems.

$$\begin{aligned} 2 \times 2 &= 2 \times S(1) \\ &= (2 \times 1) + 2 \\ &= ((2 \times 0) + 2) + 2 \\ &= (0 + 2) + 2 \end{aligned}$$

Proof Length Asymmetry

Why does proving $2 \times 2 = 4$ take 19 recursive rewriting steps in my simulation? Because proving $(0 + 2) + 2 = 4$ requires cascading applications of `add_succ` peeling both 2 variables apart simultaneously.

A key finding: Commutativity is computationally expensive! Because `mul_succ` recurses solely on the **right**:

- Proving $2 \times 1 = 2$ requires just 10 functional composing steps.
- Proving $1 \times 2 = 2$ requires 13 functional composing steps.

Mathematically they are identical. Computationally, their proof structures are entirely distinct.

5. Case Study 3: Inequality and Existentials

What Actually is an Inequality? (Concept)

How do you mathematically prove that $1 \leq 2$?

In Lean, saying $A \leq B$ simply means: **"There is some GAP number I can add to A to reach B."**

$$1 \leq 2 \implies 1 + \text{gap} = 2$$

By defining it this way, if you tell the engine what the gap is, you transform a scary inequality into a simple addition equation!

Proving Transitivity: $1 \leq 2 \wedge 2 \leq 4 \implies 1 \leq 4$

Imagine mathematical hypotheses as literal **packages** handed to you.

Step 1: Unpacking (`cases` tactic)

We use the `cases` tactic to open our packages.

- Opening the $1 \leq 2$ package gives us: a gap of **1**, and the equation $(2 = 1 + 1)$.
- Opening the $2 \leq 4$ package gives us: a gap of **2**, and the equation $(4 = 2 + 2)$.

Proving Transitivity: Packing it back up

Step 2: Packing (`use` tactic)

Now we need to prove our final goal: $1 \leq 4$.

We just have to provide the total gap! We use the `use` tactic to say: *"The new gap is just our two old gaps added together!"* ($1 + 2 = \mathbf{3}$).

Our intimidating $1 \leq 4$ goal instantly transforms into:

$$4 = 1 + \mathbf{3}$$

We then substitute the equations we unpacked to prove this equation is mathematically true!

How is $\leq$ ACTUALLY defined?

In Lean's Natural Number Game (NNG4), $a \leq b$ is not a primitive base truth. It is mathematically defined via an existential type:

$$a \leq b \iff \exists c,\ b = a + c$$

To prove an inequality, we must provide the "witness" (the numerical gap c). By doing so, we essentially translate a complex inequality problem into a simple equality problem that `rw` can solve!

In my architecture, the `use` tactic transforms the inequality goal directly into an equality by injecting this witness.

```
def use(witness, name):
    """ a <= b --> b = a + witness """
    def tactic(goal):
        a = left_of(goal)
        b = right_of(goal)
        return eq(b, add(a, witness))
    return tactic
```

The Flagship Proof: `le_trans` (Modelling Hypotheses)

Theorem: `If 1 <= 2 and 2 <= 4, then 1 <= 4`

This proof is significantly more advanced because it requires **Hypotheses**. A theorem is actually just a function taking the assumptions as arguments. Here, the arguments are `hxy: 1 <= 2` and `hyz: 2 <= 4`.

I simulated this environment by passing a `hyps` dictionary payload alongside the goal state throughout the proof timeline:

```
def proof_le_trans():
    hyps = {
        "hxy": le(one(), two()),     # Granted Hypothesis 1
        "hyz": le(two(), four()),    # Granted Hypothesis 2
    }
    goal = le(one(), four())         # Target Goal
```

Destructuring Existentials with `cases`

How do we resolve the `cases` tactic destructuring hypotheses? It executes as a data-manipulating function on the hypothesis dictionary!

When we call `cases hxy with a ha`, we are telling the proof state: "Take the fact that $1 \leq 2$, give me the gap integer (a), and give me the new guaranteed equation (ha)."

```
# Step 1: cases hxy with a ha
# Destructs (1<=2) into witness a=1, equation ha: 2=1+1
def step_cases_hxy(goal, hyps):
    hyps = dict(hyps)
    del hyps["hxy"]  # The old existential hypothesis is consumed

    # Inject the inner components of the existential back into scope
    hyps["a"]  = one()
    hyps["ha"] = eq(two(), add(one(), one()))
    return goal, hyps
```

Bridging the Gap: The `use` tactic

After extracting BOTH witnesses from the hypotheses (we got $a = 1$ from $1 \leq 2$, and $b = 2$ from $2 \leq 4$), we must combine them to prove the final goal.

To prove $1 \leq 4$, we use the combination $(a + b)$ as our new witness! Since $a = 1$ and $b = 2$, our combined witness is 3.

```
# Step 3: use a + b (combining witnesses)
def step_use_ab(goal, hyps):
    a, b = hyps["a"], hyps["b"]
    x, z = left_of(goal), right_of(goal)

    # Inequality Goal (1 <= 4) structurally transforms into:
    # 4 = 1 + (1 + 2)
    return eq(z, add(x, add(a, b))), hyps
```

Hypotheses as Runtime Rewrite Rules

Our goal is now `4 = 1 + (1 + 2)`. How do we verify this? We rewrite the Goal using the actual mathematical data stored inside the hypotheses!

```
# Step 4: rw [hb]
# Using the hypothesis ‘hb: 4=2+2‘ to rewrite 4 in the goal
def step_rw_hb(goal, hyps):
    # Goal transitions from: 4 = ... to: (2 + 2) = ...
    return eq(add(two(), two()), rhs_of(goal)), hyps

# Step 5: rw [ha]
# Using the hypothesis ‘ha: 2=1+1‘ to rewrite the leftmost 2
def step_rw_ha(goal, hyps):
    # Goal transitions to: ((1 + 1) + 2) = 1 + (1 + 2)
    lhs = lhs_of(goal)
    return eq(add(add(one(), one()), right_of(lhs)), rhs_of(goal)), hyps

# Step 6: exact add_assoc
# Terminate by applying the mathematically proven theorem (a+b)+c = a+(b+c)
def step_add_assoc(goal, hyps):
    return eq(rhs_of(goal), rhs_of(goal)), hyps
```

6. Insights into the Lean 4 Compiler Architecture

The Two-Tier Compiler Pipeline

My research into Lean's internals revealed a highly secure separation of concerns:

```
    Source Code (.lean)
           |
      [ PARSER ]     Text -> Concrete Syntax Tree
           |
    [ ELABORATOR ]   Syntax -> Core `Expr`. (Where Tactics run!)
           |         Runs `MetaM` Monad, Unification, MetaVariables.
           |         MASSIVE codebase (500,000+ lines). Expected to have
                bugs.
           |
      [ KERNEL ]     The Trusted Core (~5k lines C++).
           |         Type-checks the generated proof term.
           v
   [ ENVIRONMENT ]   Stores mathematical ground truth permanently.
```

The de Bruijn Criterion & Complete Isolation

Why is the architecture bifurcated this way?

- **Untrusted Zone:** Tactics, Macros, Parsing, Unification. Allowed to crash, infinite-loop, or have severe logical bugs.
- **Trusted Kernel:** Tiny, mathematically pure type-checker written in memory-safe C++. Analyzes pure ASTs.
- If an AI or a buggy tactic generates a fake functional composition, the kernel strictly **rejects the payload** with a type error. (e.g. `Array Int (2+2)` failing to reduce against `Array Int (4)`).

The Type Checker & Definitional Computation

Checking types in Dependent Type Theory is Turing-Complete.
The kernel evaluates expressions via **Definitional Equality (`isDefEq`)**.

It features 4 explicit reduction engine sub-systems:

- β **(Beta) Reduction**: Function execution / lambda evaluation.
- δ **(Delta) Reduction**: Unfolding definitions (`double 2` $\rightarrow$ `2 + 2`).
- ζ **(Zeta) Reduction**: Resolving let-bindings.
- ι **(Iota) Reduction**: Executing Inductive Type **Recursors** (Pattern matching).

Conclusion

Conclusion: The Ultimate Synthesis

FP Concept	Lean Architecture Equivalent
Algebraic Data Types	Tagged Unions (e.g. `("add", a, b)`) or `Expr`
Immutability	Modifying proofs returns strictly new nodes
Pattern Matching	Destructing Tuples, executing ι-reduction
Pure Functions	`goal` $\rightarrow$ `goal` with zero side effects
Higher-Order Functions	Tactics returning configuration closures
Type Systems	The Curry-Howard Theorem Verification boundary

Lean operates beautifully on a single mathematical principle:
Proving a mathematical theorem IS exactly the compilation of a verified program.

Questions?

Thank you for your time.